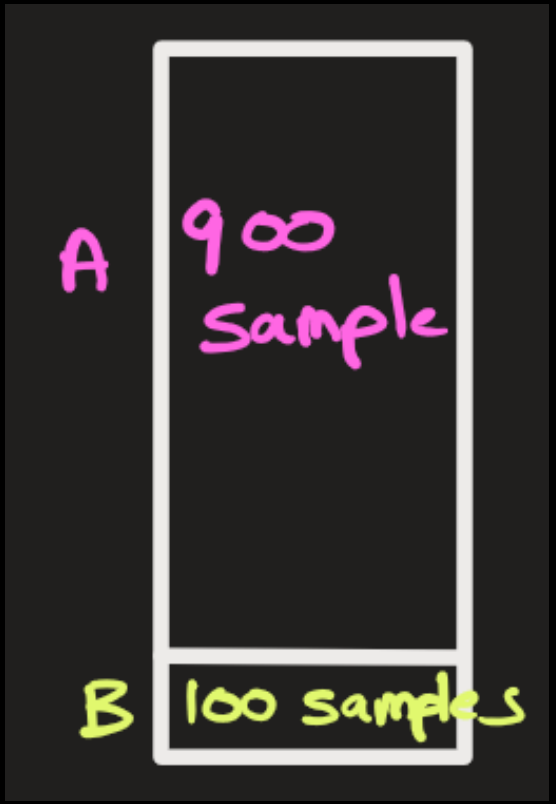
Imbalanced Data

What is Imbalanced Data?

Imbalanced data means **one class has significantly more samples than another class.**

Patient_id	Test1	Test2	Cancer
1	0.2	0.4	N
2			N
3			N
…	…	…	…
900	0.5	0.8	N
901	0.4	0.1	Y
…	…	…	…
1000	0.7	0.9	Y



Real-Life Example

Suppose:

- 10,000 transactions
- 9,900 normal
- 100 fraud

Fraud = Only 1%

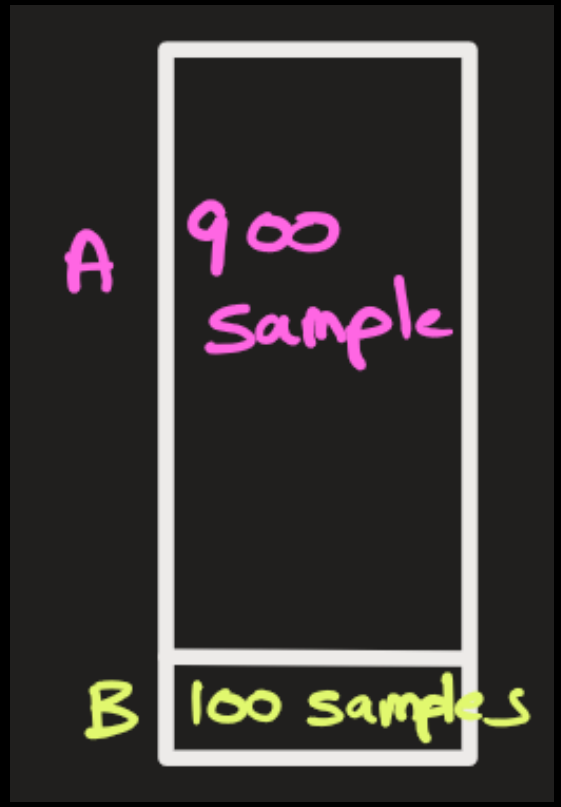
Real-Life Example

Suppose:

- 1000 Patients
- 900 normal
- 100 serious disease

Disease = Only 10%

Problem	Majority Class	Minority Class
Fraud Detection	Normal Transactions	Fraud
Medical Diagnosis	Healthy	Disease
Spam Detection	Non-Spam	Spam



Why is Imbalanced Data Dangerous?

Suppose:

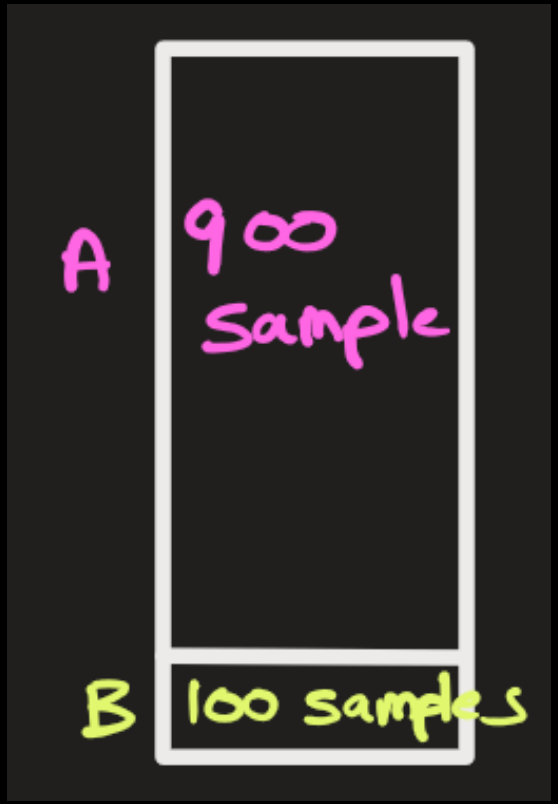
- 10,000 transactions
- 9,900 normal
- 100 fraud

Fraud = Only 1%

If model predicts “Every sample is Normal”, then
accuracy = 9900 / 10000 = 99%

Looks great, **but model is useless because it never detects fraud**.

Accuracy is misleading in imbalanced datasets.



Accuracy is misleading in imbalanced datasets.

Important Evaluation Metrics

- **Precision**

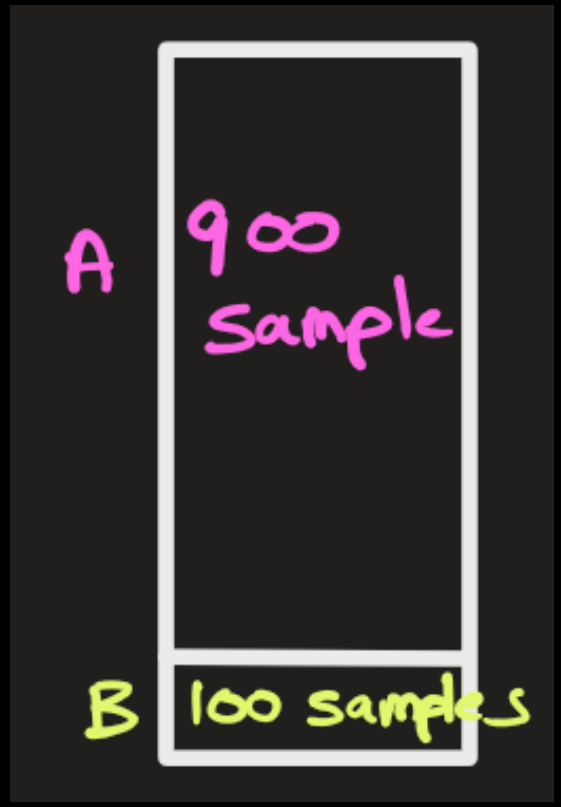
Out of predicted positive, how many are correct?

Precision = TP / (TP + FP)

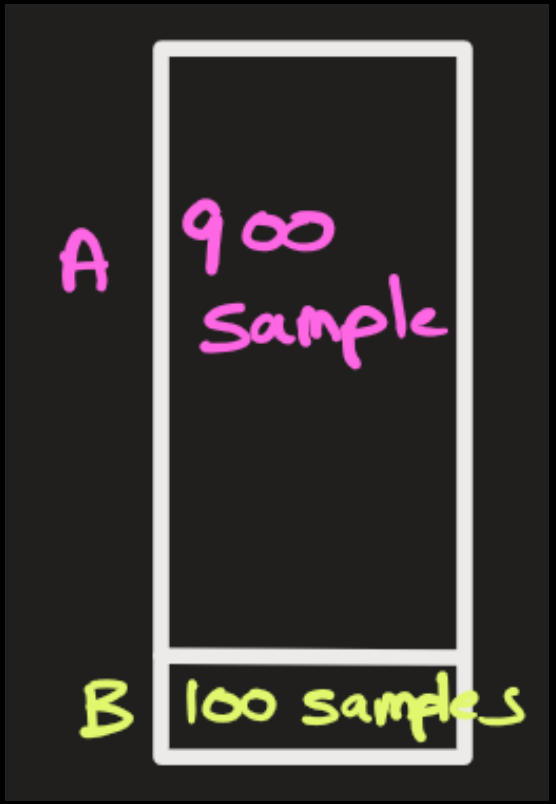
- **Recall (Very Important for Imbalance)**

Out of actual positive, how many detected?

Recall = TP / (TP + FN)



Techniques to Handle Imbalanced Data



Approach 1: Data Level Methods

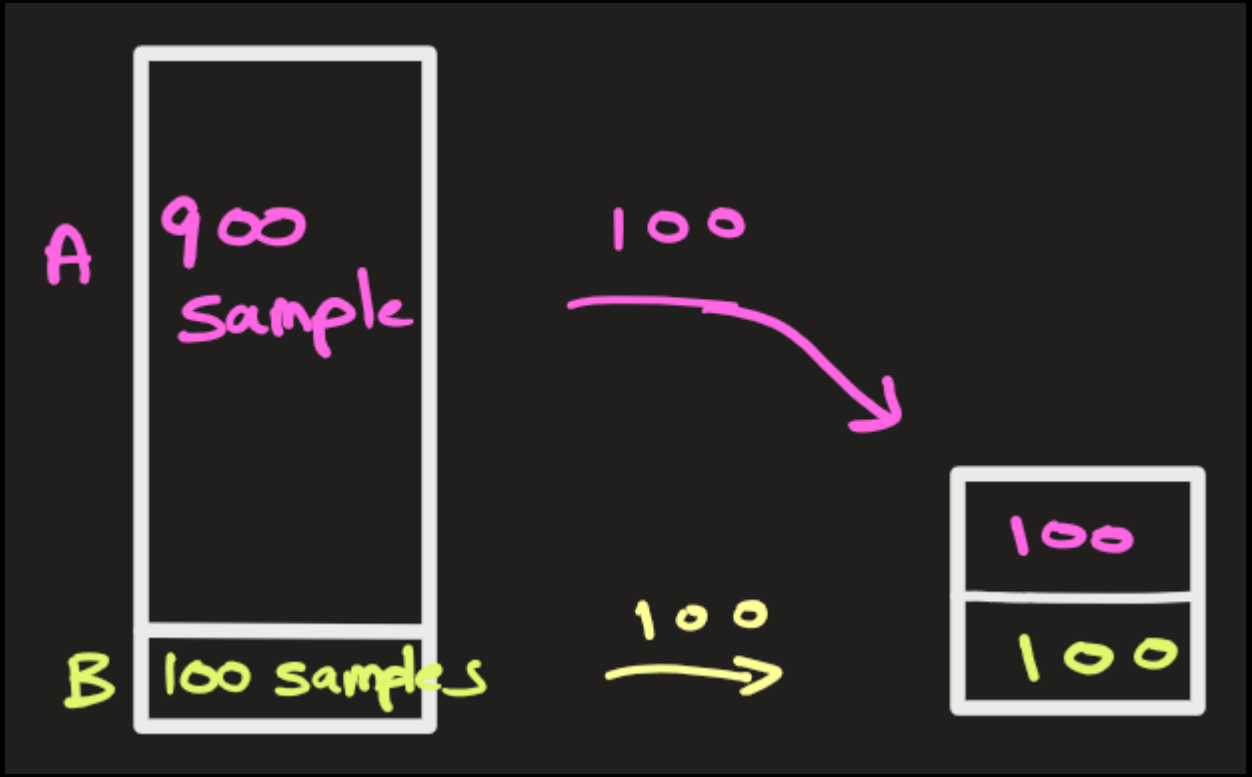
(Modify dataset)

1. Random Undersampling:
Remove samples from majority class.
Here you choose only a small subset of samples from majority class.

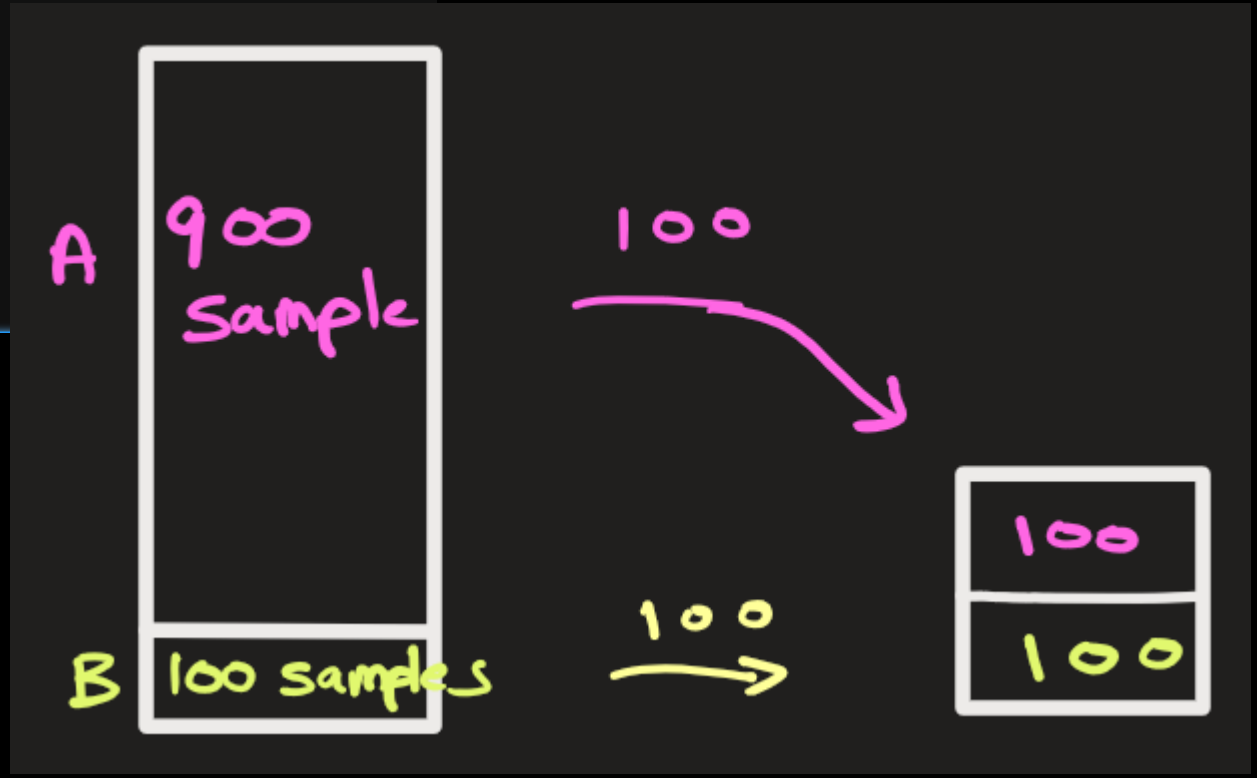
Advantage:
- Fast
- Simple

Disadvantage:
- Loose useful data

because you are throwing away lot of data



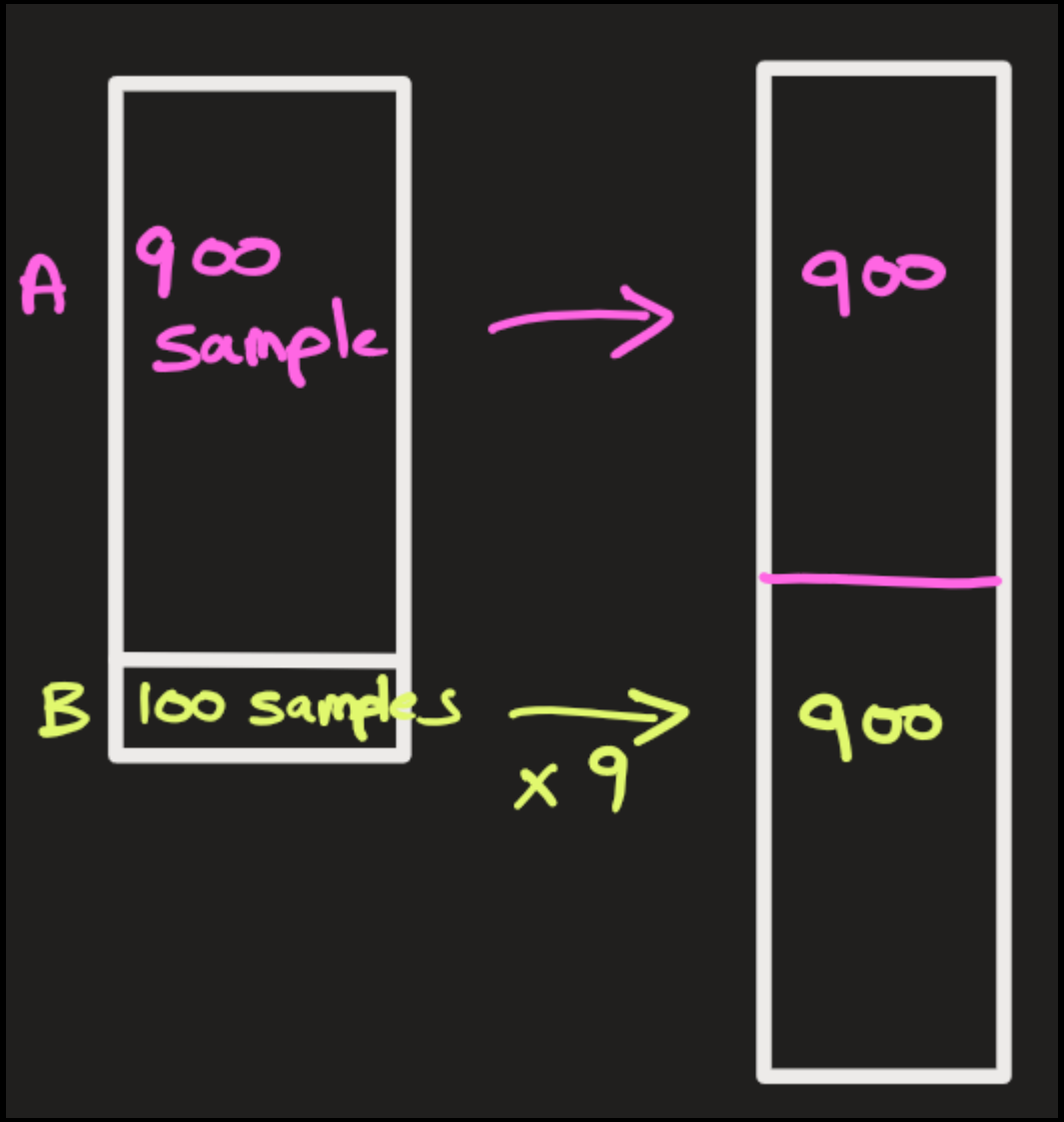
```python
from imblearn.under_sampling import RandomUnderSampler
from collections import Counter
import pandas as pd

df = pd.read_csv("some_file.csv")
X = df[['test1', 'test2', 'test3']]
y = df['disease']

rus = RandomUnderSampler()
X_res, y_res = rus.fit_resample(X, y)

print(Counter(y_res))
```



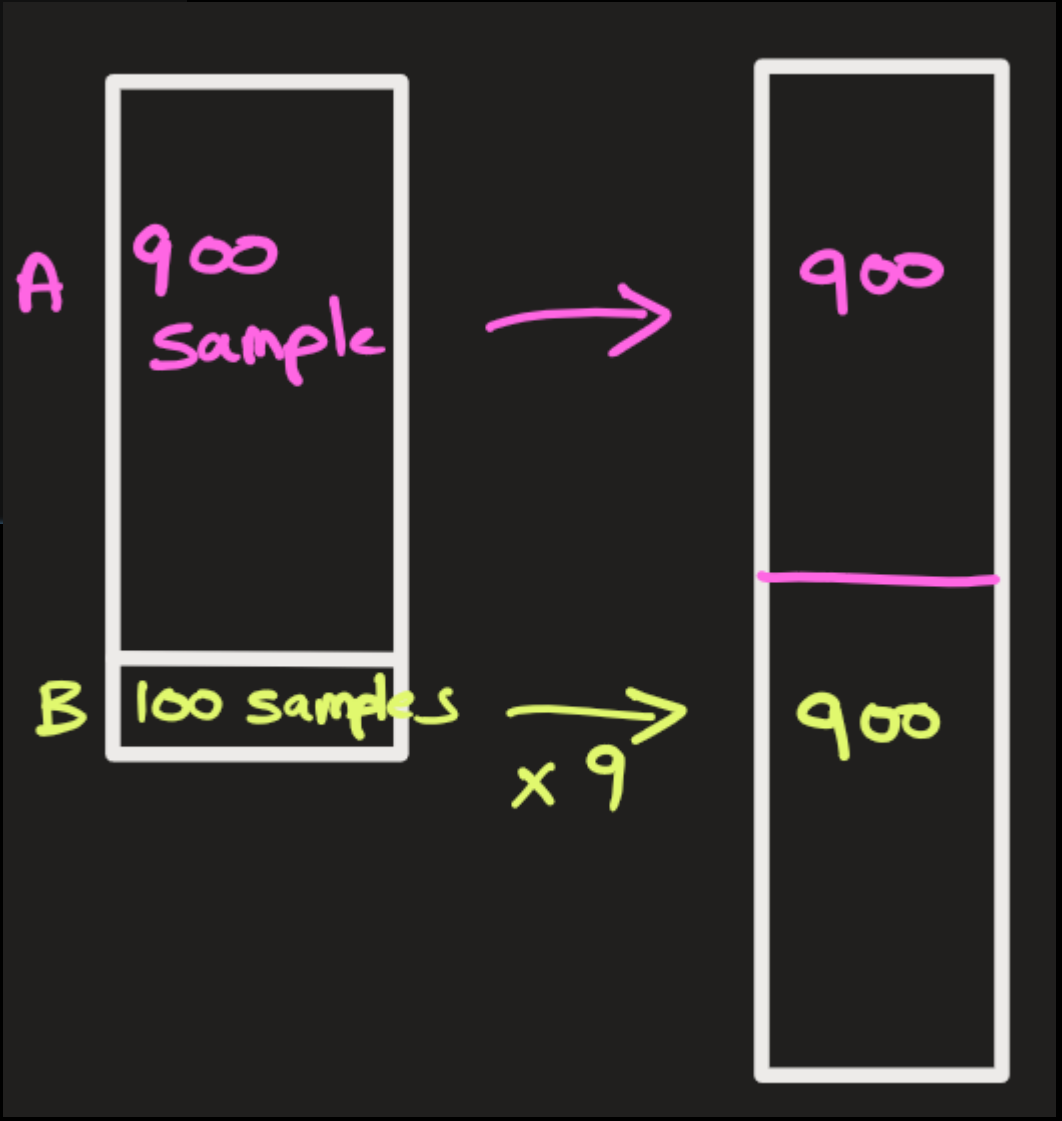
2. Random Oversampling

Duplicate minority samples.

Problem:
Overfitting may happen.



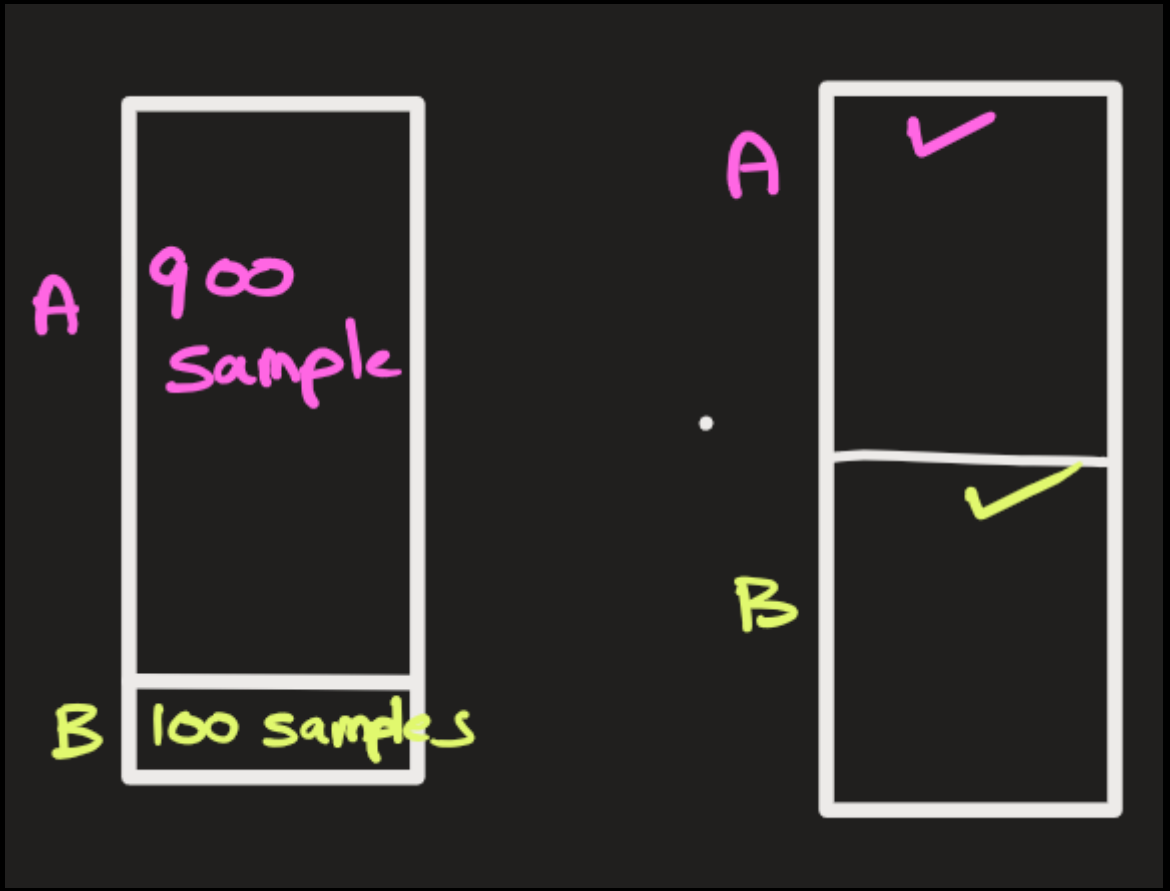
```python
from imblearn.over_sampling import RandomOverSample
import pandas as pd

df = pd.read_csv("some_file.csv")
X = df[['test1', 'test2', 'test3']]
y = df['disease']

ros = RandomOverSampler()
X_res, y_res = ros.fit_resample(X, y)
```



3. SMOTE (Most Important)

Synthetic Minority Over-sampling Technique

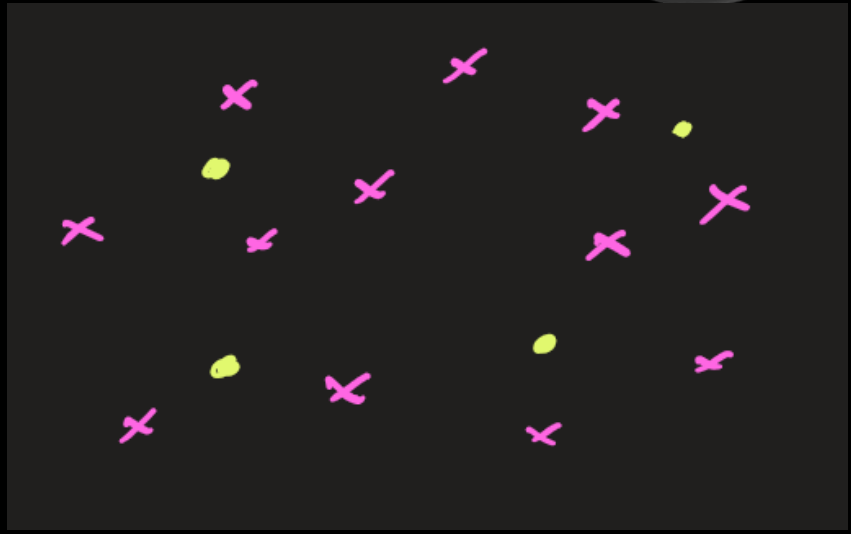
- Creates artificial minority samples
- Does NOT duplicate data
- Creates new realistic samples



SMOTE

Visual Intuition

Think of minority points as **dots in space**.



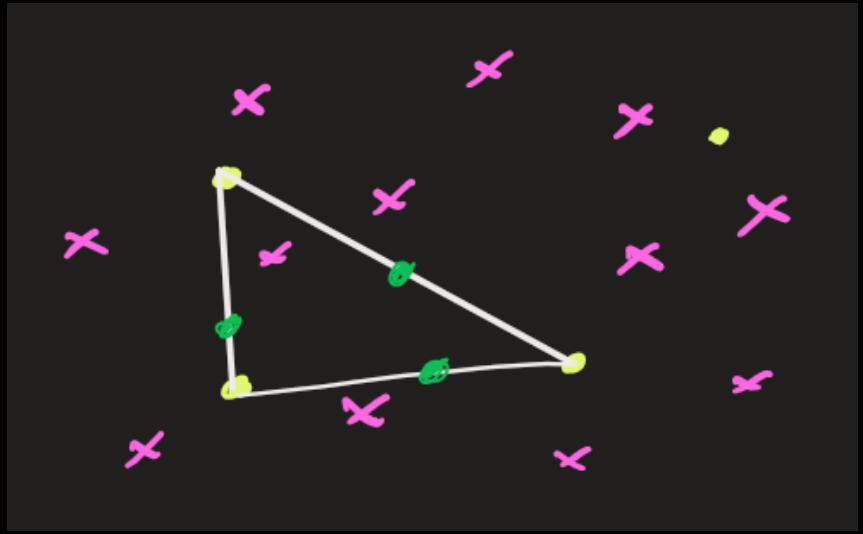
SMOTE:

Step1) Draws imaginary lines between **nearby minority** dots

It uses KNN to find nearby points

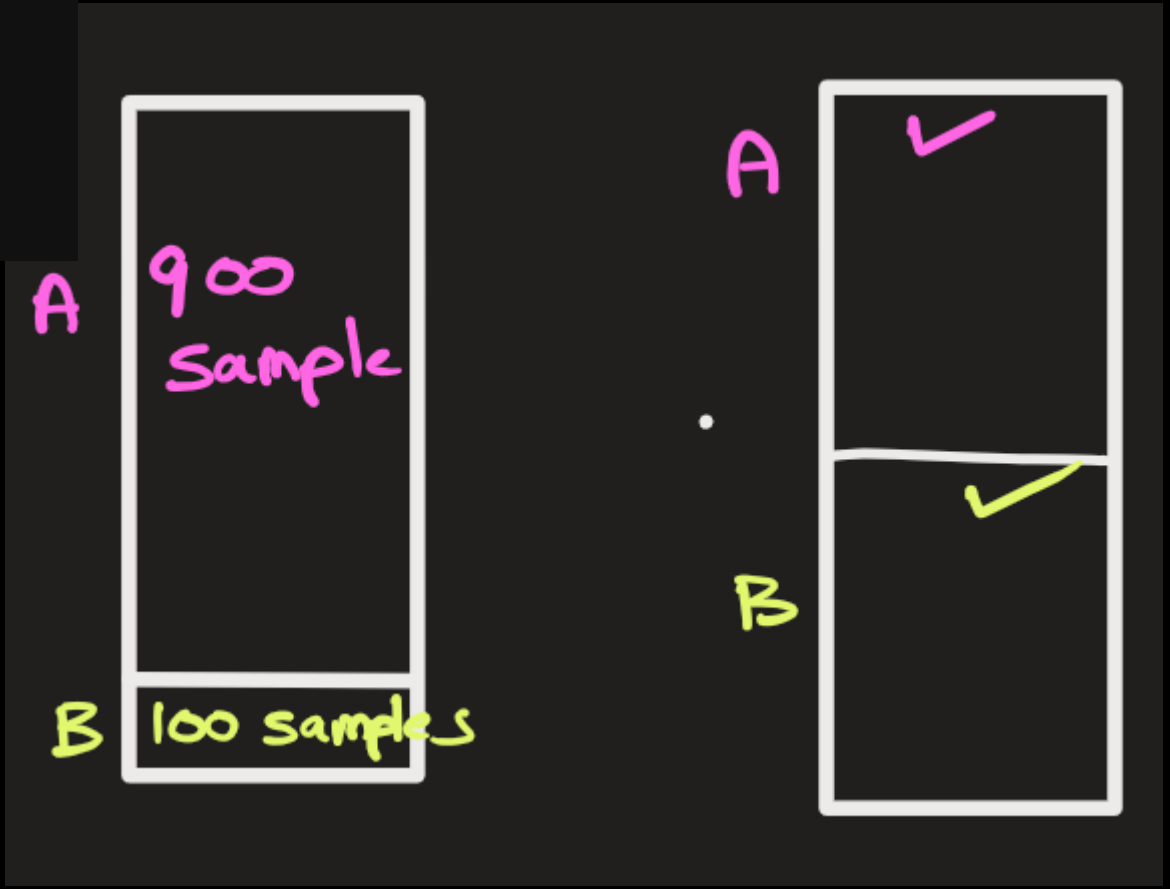
Step2) Randomly places new dots **on those lines**

These new dots (shown green) are new samples



This **expands the minority region** instead of stacking duplicates.

```python
from imblearn.over_sampling import SMOTE
import pandas as pd

df = pd.read_csv("some_file.csv")
X = df[['test1', 'test2', 'test3']]
y = df['disease']

smote = SMOTE()
X_res, y_res = smote.fit_resample(X, y)
```

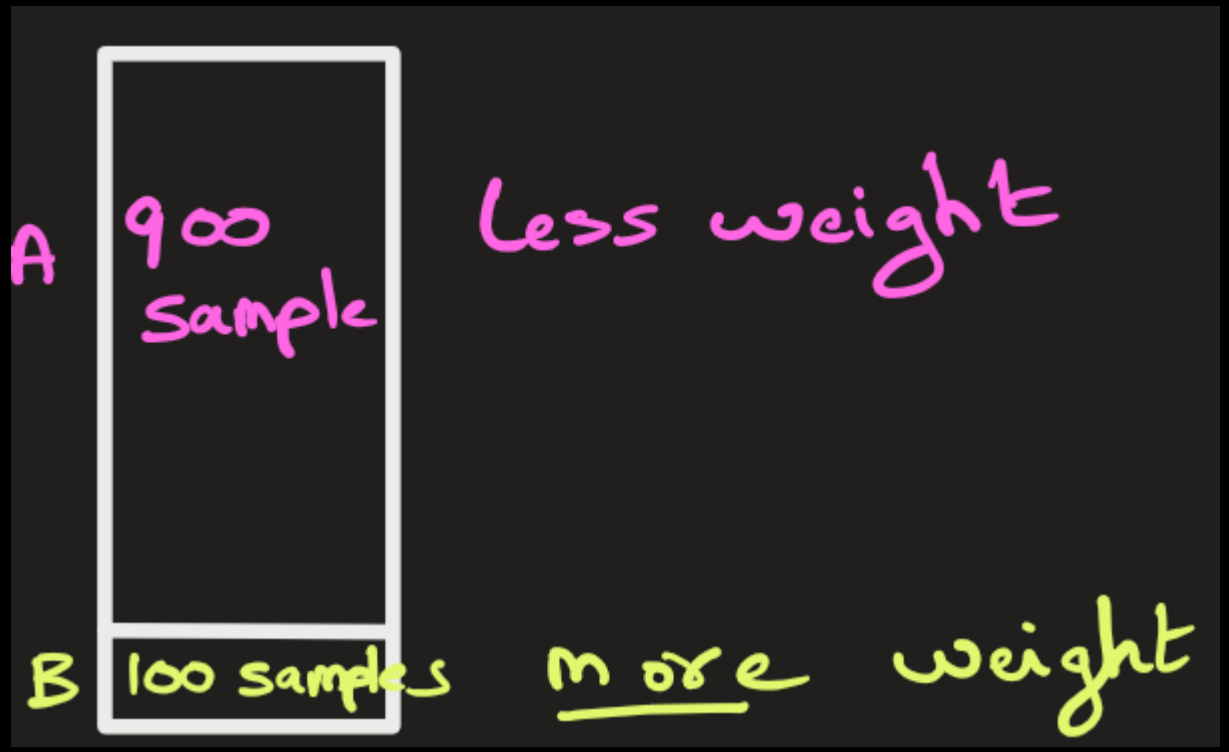


Algorithm Level Methods

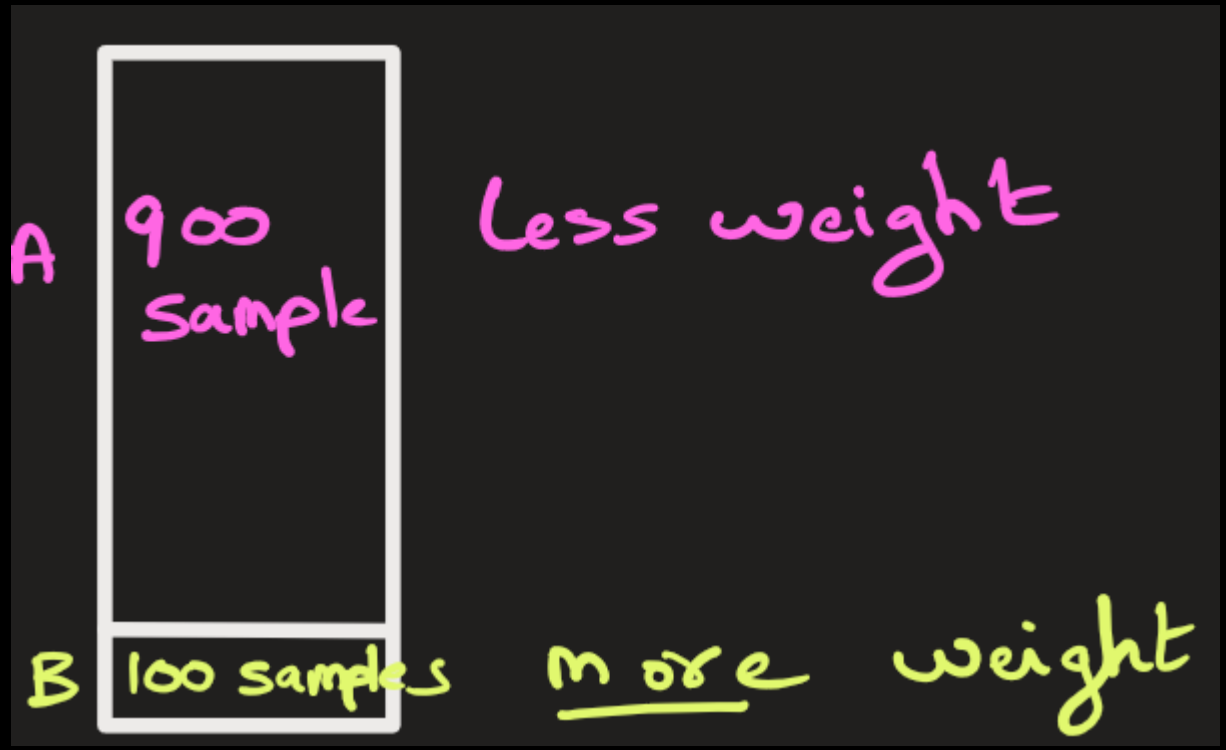
(Modify model)

1. Class Weighting

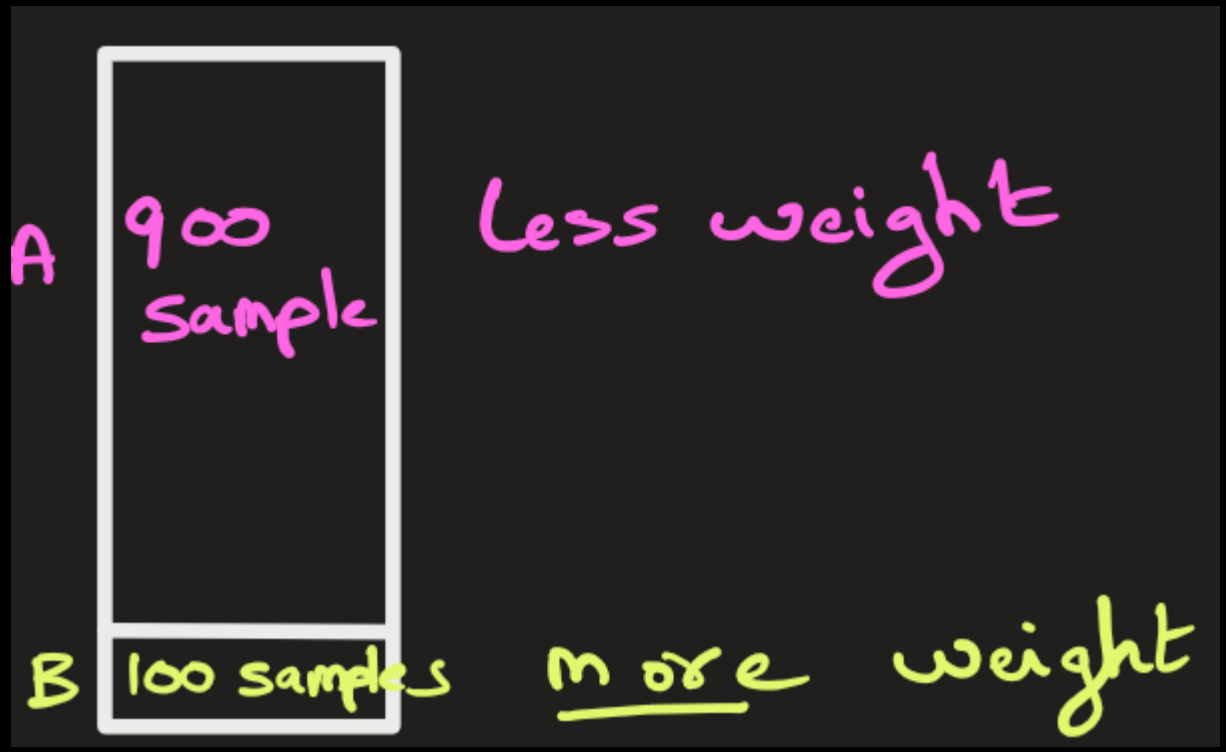
Give higher importance to minority class.



```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced')
model.fit(X_train, y_train)
```

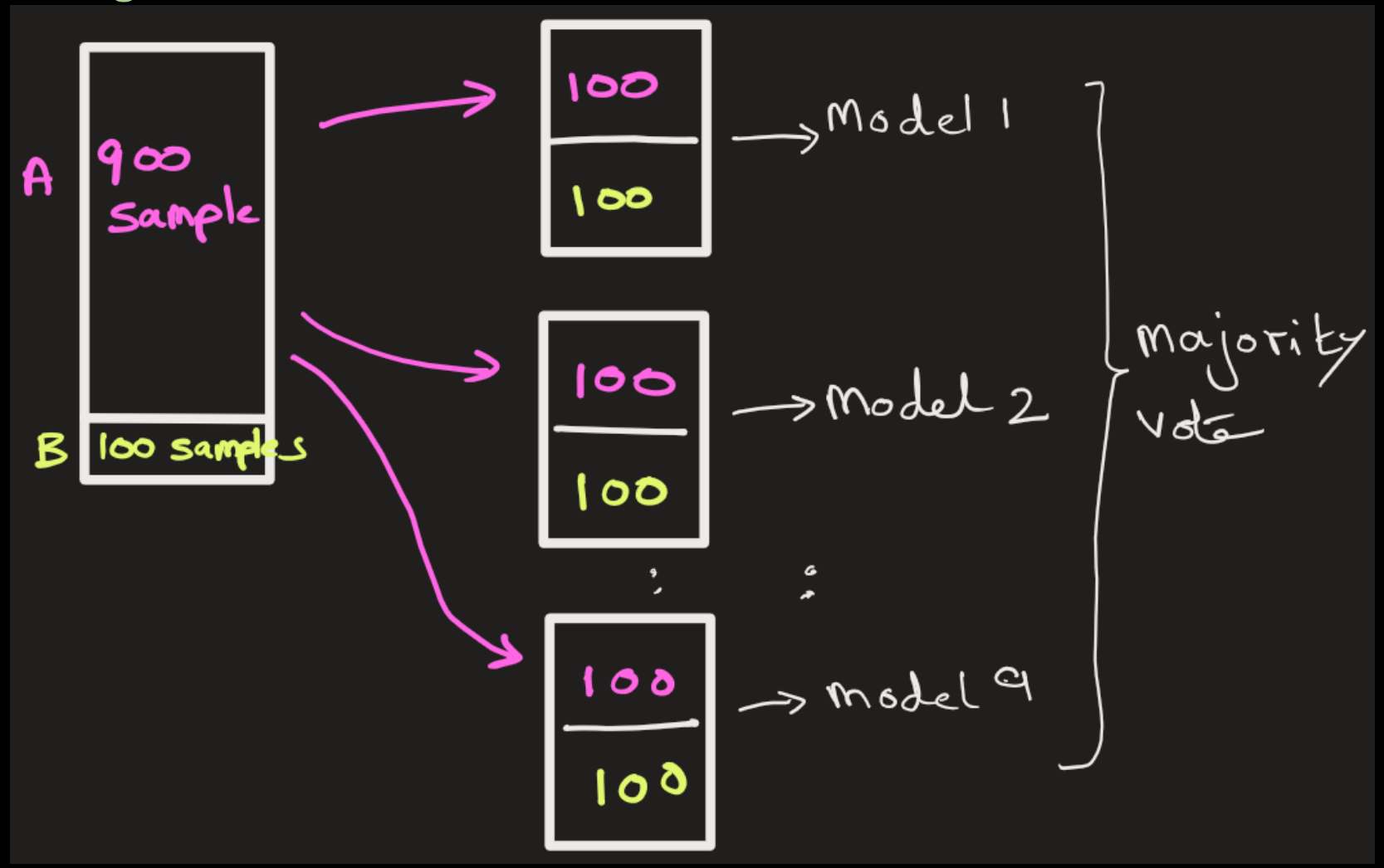


```python
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(class_weight='balanced')
model.fit(X_train, y_train)
```



2. Ensemble Techniques (Very Powerful)

Build multiple balanced datasets and then build models on them.
Then use majority vote for inferencing



```python
from imblearn.ensemble import BalancedRandomForestClassifier

model = BalancedRandomForestClassifier()
model.fit(X_train, y_train)
```

Heading Goes Here

Fhdsklf
Fjdsklf
Fjskldf